



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



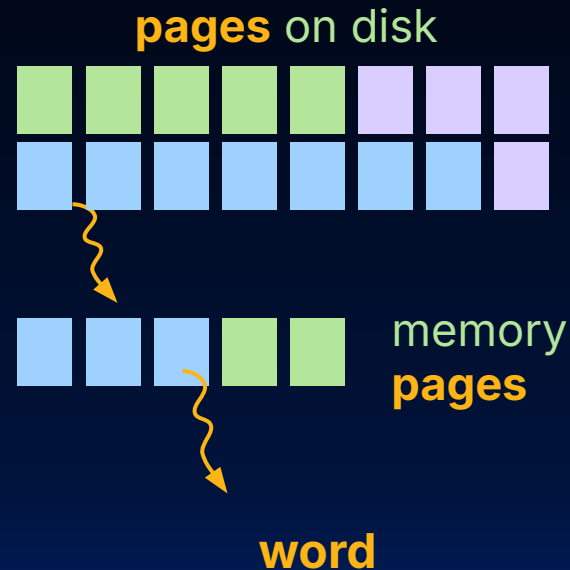
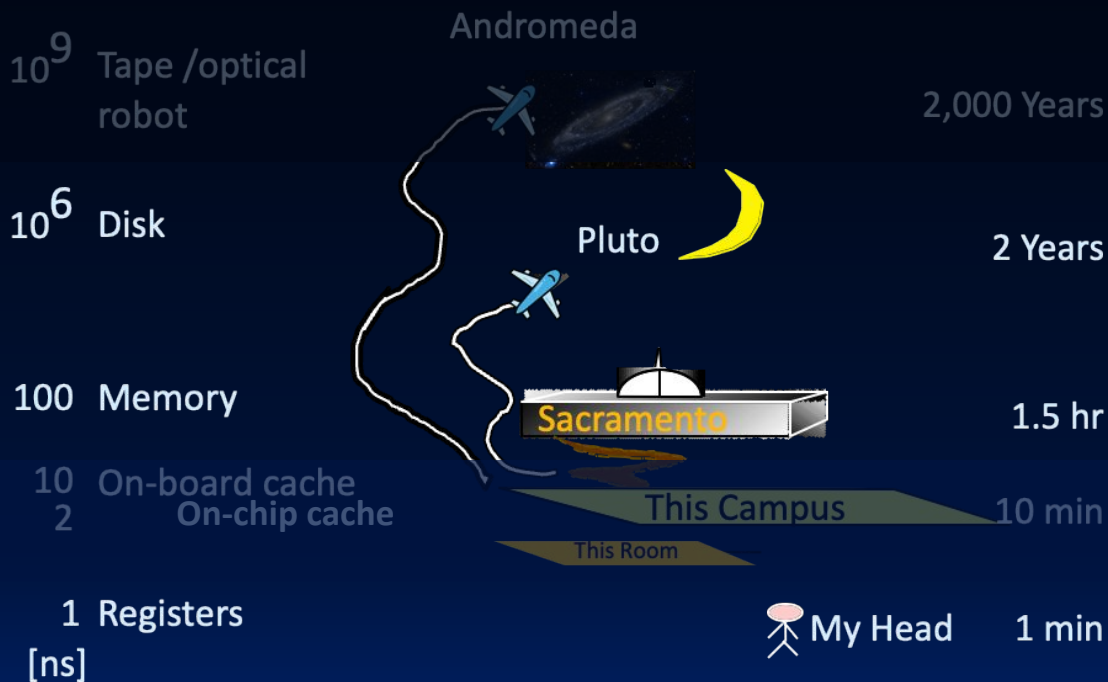
Assistant
Teaching Professor
Lisa Yan

Virtual Memory I

Disk to Memory: Pages

How Far Away is the Data?

Spatial/temporal locality:
Store likely-to-access data on memory. Fetch from disk sparingly.



A disk access loads an entire **page** into memory.



Today: Translate to Provide Illusion of Storage

RV32I provides a 32-bit address space.

→ 2^{32} B = 4 GiB addressable memory

0xFFFF FFFF

0x0000 0000

Virtual address space

what program thinks it has

Suppose **RAM** is 1GiB.

→ 2^{30} B addressable memory.

Physical address space
physical location in memory (DRAM)

- Virtual memory helps two cases:
- [for now] Uniform storage illusion
 - [next time] Protection between multiple processes



Address Translation

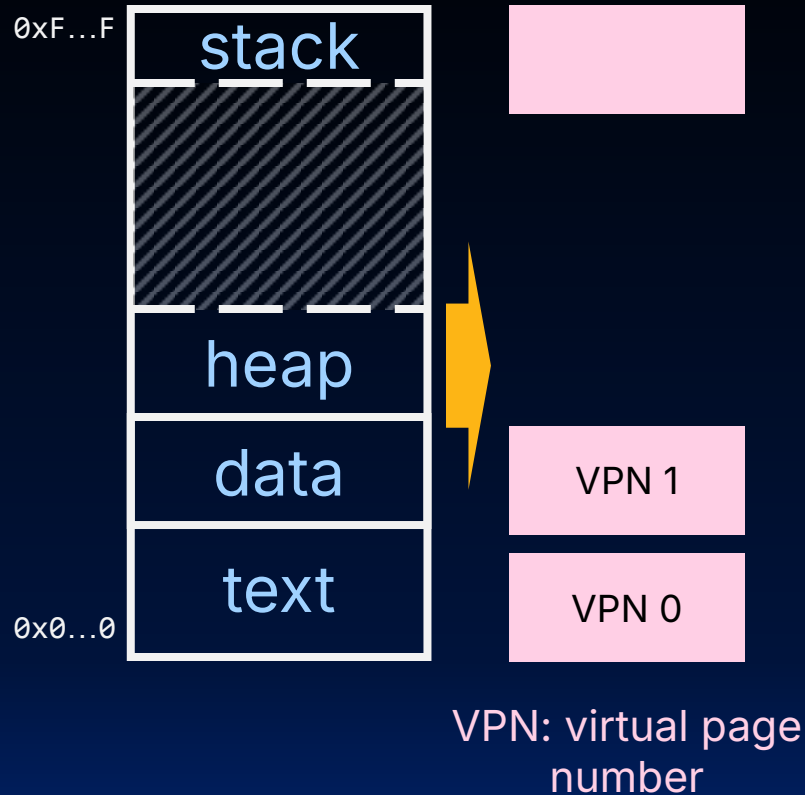
- Address Translation
- Address Translation Practice
- Page Table Details I
- Memory/Disk Access



Paged Memory (1/2)

- The concept of "**paged memory**" dominates:
- Typical page size: **4-16 KiB**

```
$ cat /proc/cpuinfo
$ getconf PAGESIZE # 4096
```





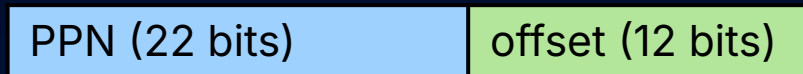
Address Translation

- Memory address translation maps a **Virtual Page Number (VPN)** to a **Physical Page Number (PPN)**.

Virtual address (e.g. 64 bits)



Physical address (e.g. 34 bits)



To address 4KiB bytes on page, need 12 bits of **page offset**

Virtual and physical pages are **the same size**. Same # bits to address offset within a page!

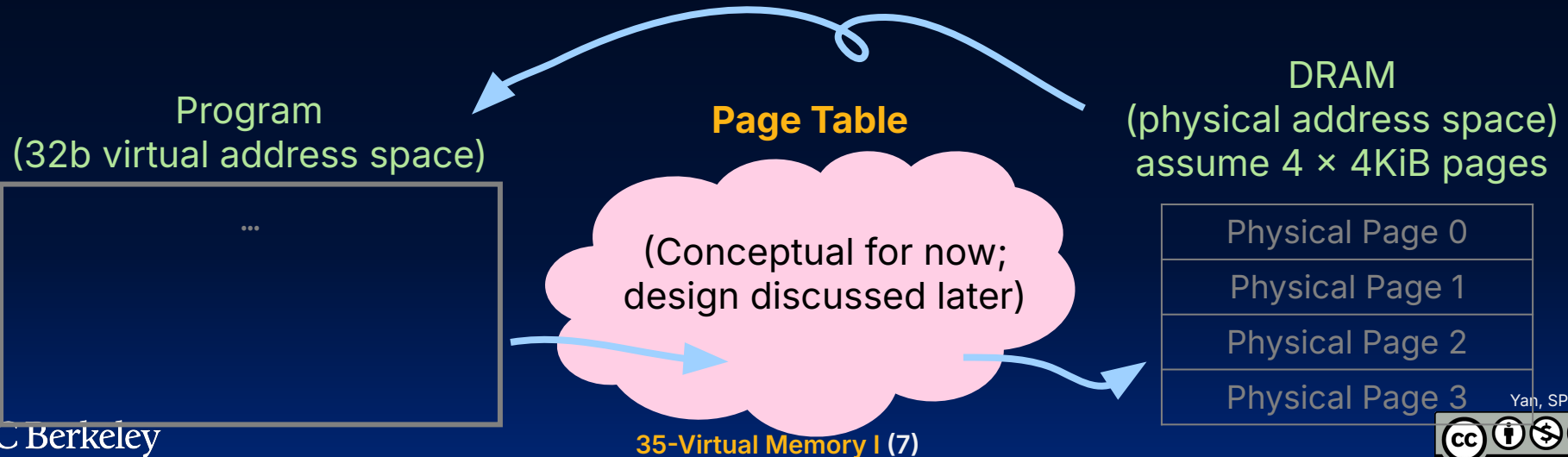
DRAM/memory
(physical address space)
assume $4 \times 4\text{KiB}$ pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation

- A **page table** translates virtual addresses to physical addresses for a given process.
 - Each entry in the table corresponds to a virtual page number.
 - If page is in memory, entry has corresponding physical page number.
 - Else, entry should tell OS to trigger **page fault** to **load page from disk**.





Address Translation I: Page Is in Memory (1/4)

1. Program executes a load specifying a **virtual address (VA)**.

Program
(32b virtual address space)

Load byte @ **0xFFFF F004**
to register t0

1

Page Table



35-Virtual Memory I (8)

DRAM
(physical address space)
assume $4 \times 4\text{KiB}$ pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation I: Page Is in Memory (2/4)

2

OS translates **VA** to the **physical address (PA)** in memory.

- Extract virtual page number (VPN) from VA
- Look up physical page number (PPN) in **page table**
- Construct PA: physical page number + offset (from virtual address)

Program
(32b virtual address space)

Load byte @ 0xFFFF F004
to register t0

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1

2

35-Virtual Memory I (9)

DRAM

(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation I: Page Is in Memory (3/4)

3

Page Fault: If ~~physical page~~ not in memory, then OS loads page from disk.

Page table entry is valid!
No page fault because
page is in memory.

Program
(32b virtual address space)

Load byte @ 0xFFFF F004
to register t0

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1



3

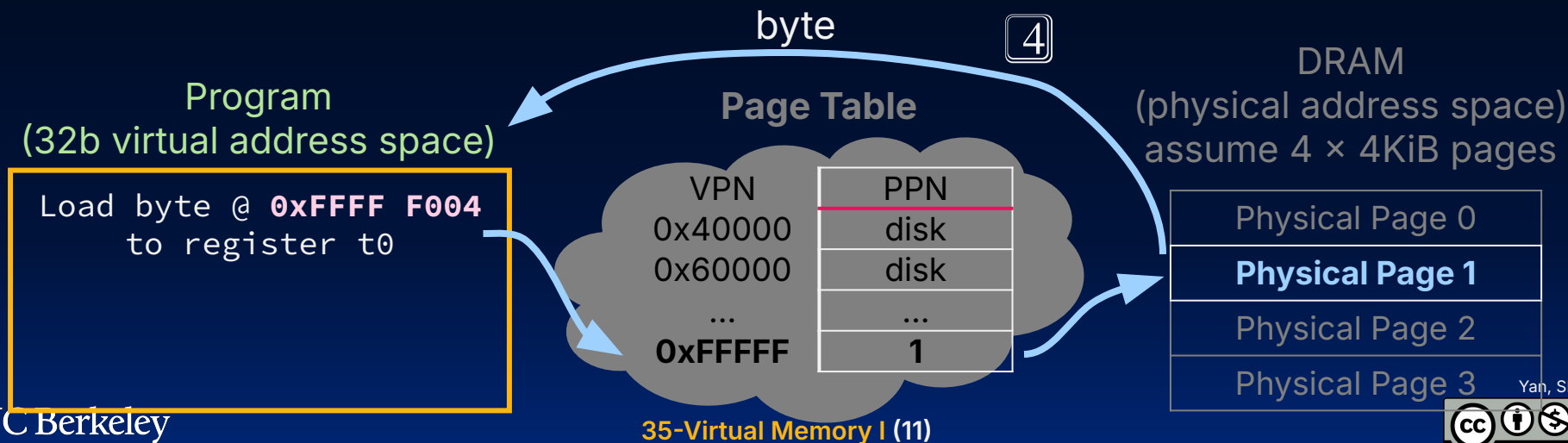
DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation I: Page Is in Memory (4/4)

4. Read memory at the **physical address** and return the data to the program.





Address Translation I: Page Is Not in Memory (1/4)

1. Program executes a load specifying a **virtual address (VA)**.

Program
(32b virtual address space)

```
Load byte @ 0xFFFF F004  
to register t0  
Load byte @ 0x6000 0030  
to register t0
```

1

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1

35-Virtual Memory I (12)

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26



Address Translation I: Page Is Not in Memory (2/4)

2

OS translates **VA** to the **physical address (PA)** in memory.

- Extract virtual page number (VPN) from VA
- Look up physical page number (PPN) in **page table**
- Construct PA: physical page number + offset (from virtual address)

Program
(32b virtual address space)

```
Load byte @ 0xFFFF F004  
to register t0  
Load byte @ 0x6000 0030  
to register t0
```

2

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1

(still conceptual
image)

DRAM

(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation I: Page Is Not in Memory (3/4)

3

Page Fault: If **physical page** not in memory, then OS loads page from disk.

Page fault! Need to load page from disk.

Program
(32b virtual address space)

```
Load byte @ 0xFFFF F004
to register t0
Load byte @ 0x6000 0030
to register t0
```

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1



3

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

Address Translation I: Page Is Not in Memory (3/4)

3

Page Fault: If **physical page** not in memory, then OS loads page from disk.

Page fault! Need to load
page from disk.
(a million cycles later)

Program
(32b virtual address space)

```
Load byte @ 0xFFFF F004
to register t0
Load byte @ 0x6000 0030
to register t0
```

Page Table

VPN	PPN
0x40000	disk
0x60000	2
...	...
0xFFFFF	1



3

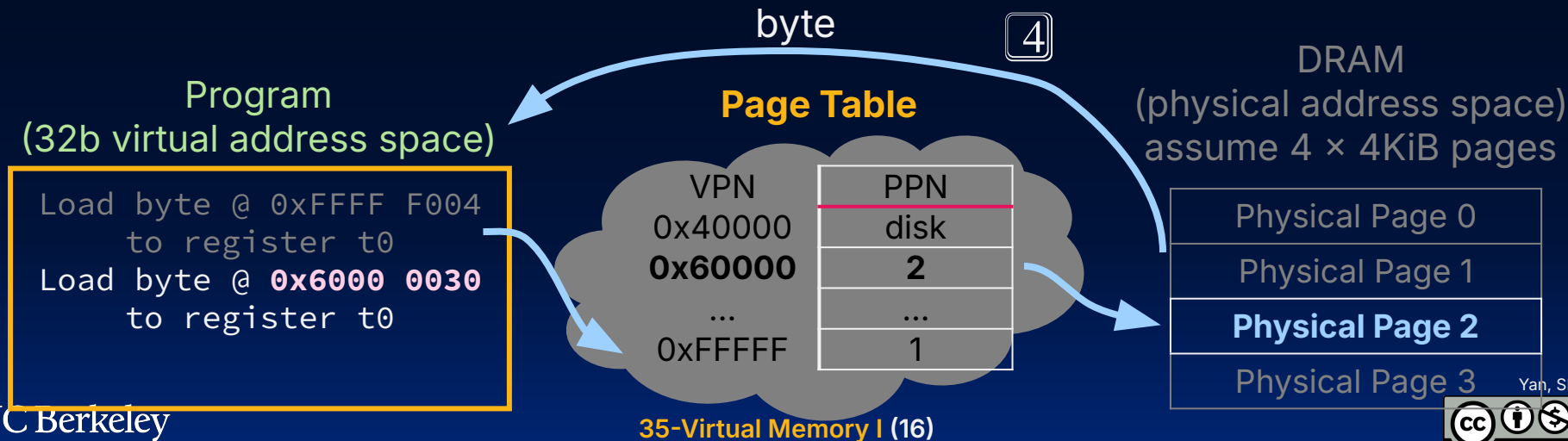
DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Yan, SP26

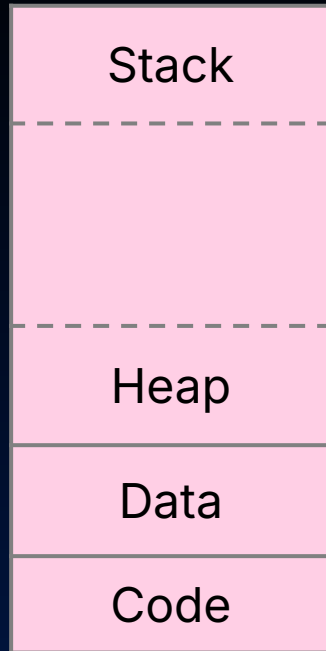
Address Translation I: Page Is Not in Memory (4/4)

4. Read memory at the **physical address** and return the data to the program.



What you just saw: Demand Paging (1/2)

- Loading a ton of pages for every new program is wasteful.
 - What if the program is just "hello world"?



Our VM abstraction allows a program to use the full virtual address space...



...but some programs use only a tiny amount of memory.

What you just saw: Demand Paging (2/2)

- Loading a ton of pages for every new program is wasteful.
 - What if the program is just "hello world"?

- Solution: **Demand paging**.
 - Only load a page into memory if the **user requests it**.
 - When a program starts, the page table says "**Disk**" for every entry.

Process 1's Page Table	
VPN	PPN
0x00001	Disk
0x00002	Disk
0x00003	Disk
0x00004	Disk
...	...

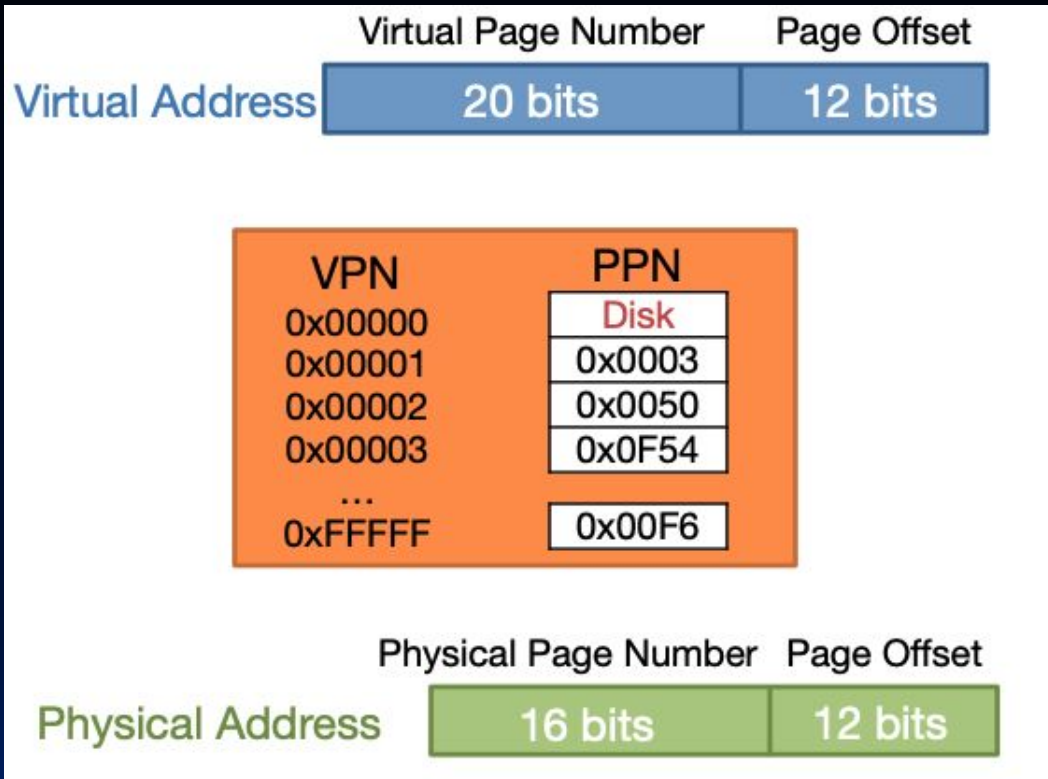
Unused
Stack
Unused
Heap
Unused
Data
Unused
Code
Unused



Address Translation Practice

- Address Translation
- Address Translation Practice
- Page Table Details I
- Memory/Disk Access

Translation Example



What Physical Address does this Virtual Address translate to?

0x00003450

- A. 0x00003450
- B. 0x0000250
- C. 0x00503450
- D. 0x0F543450
- E. 0x0F54450
- F. Disk/Other



What Physical Address does Virtual Address 0x00003450 translate to?

	Virtual Page Number	Page Offset
Virtual Address	20 bits	12 bits

VPN	PPN
0x00000	Disk
0x00001	0x0003
0x00002	0x0050
0x00003	0x0F54
...	
0xFFFFF	0x00F6

	Physical Page Number	Page Offset
Physical Address	16 bits	12 bits

A. 0x00003450

0%

B. 0x0000250

0%

C. 0x00503450

0%

D. 0x0F543450

0%

E. 0xF54450

0%

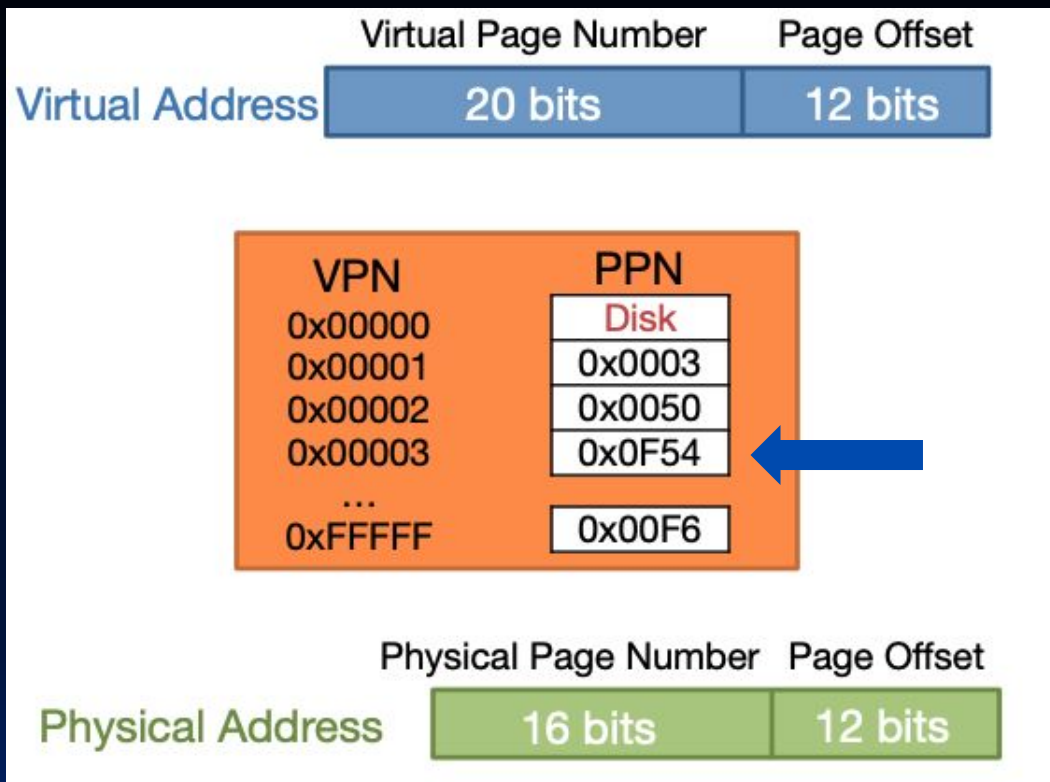
F. Disk/Other

0%



SP21

Translation Example: Solution



What Physical Address does this Virtual Address translate to?

0x00003450

A. 0x00003450

B. 0x0000250

C. 0x00503450

D. 0x0F543450

E. 0x0F54450

F. Disk/Other

0x0F54450



Page Table Details I

- Address Translation
- Address Translation Practice
- Page Table Details I
- Memory/Disk Access



Setup

- Assume a 32-bit machine with 8GiB of RAM and 16KiB pages.
- How many bits would there be for each of the following?

Virtual address

VPN

offset

Physical address

PPN

offset

- | | A. | B. | C. | D. | E. | F. |
|--|----|----|----|----|----|-------|
| 1. Page offset | 14 | 15 | 16 | 19 | 20 | Other |
| 2. VPN | 14 | 15 | 16 | 19 | 20 | Other |
| 3. PPN | 14 | 15 | 16 | 19 | 20 | Other |
| 4. Suppose each entry in the page table is 4B.
How large is the page table, in MiB? | | | | | | |



How many bits would there be for each of the following?

- Assume a 32-bit machine with 8GiB of RAM and 16KiB pages.
- How many bits would there be for each of the following?

Virtual address



Physical address



A. B. C. D. E. F.

1.	Page offset	14	15	16	19	20	Other
2.	VPN	14	15	16	19	20	Other
3.	PPN	14	15	16	19	20	Other



26



Setup: Solution

- Assume a **32-bit** machine with **8GiB of RAM** and **16KiB pages**.
- How many bits would there be for each of the following?

Virtual address

VPN

offset

Physical address

PPN

offset

- Page offset $= \log_2(16 \text{ KiB}) = \log_2(2^4 * 2^{10}) = 14 \text{ bits}$
- VPN $= 32 - 14 = 18 \text{ bits}$
- PPN $= \log_2(8 \text{ GiB}) - 14$
 $= \log_2(2^3 * 2^{30}) - 14 = 33 - 14 = 19 \text{ bits}$



Setup: Continued

- Assume a **32-bit** machine with **8GiB of RAM** and **16KiB pages**.
- How many bits would there be for each of the following?
 - Page offset: 14 bits
 - VPN: 18 bits
 - PPN: 19 bits

Virtual address

VPN

offset

Physical address

PPN

offset

- Suppose each entry in the page table is 4B.
How large is the page table, in MiB?
(1 entry per VPN)

$$\begin{aligned} &2^{18} \text{ entries} \times 4 \text{ B/entry} \\ &= 2^{20} \text{ B} = \mathbf{1 \text{ MiB}} \end{aligned}$$



Memory/Disk Access

- Address Translation
- Address Translation Practice
- Page Table Details I
- Memory/Disk Access

What *are* Page tables?

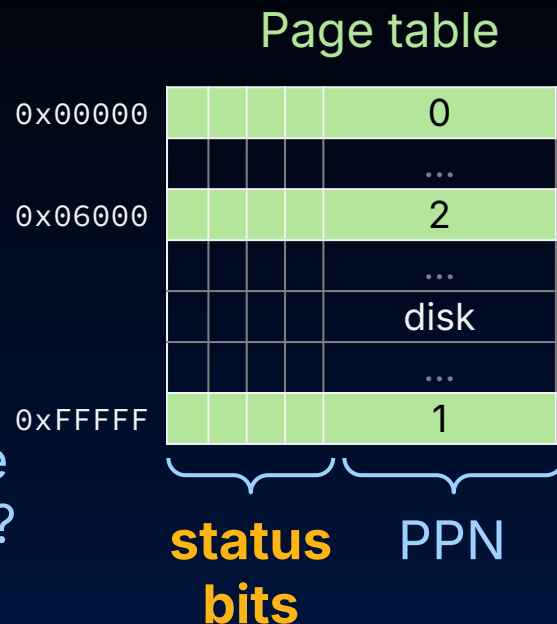
- One page table per process.
- One entry per virtual page number:
 - Entry has **physical page number** (if in memory)
 - Entry has **status bits**: valid, dirty, write protection (discussed later)

1. Is a page table a type of cache?

- A.** Yes
- B.** No

2. Can a full page table be stored in the cache?

- A.** Yes
- B.** No





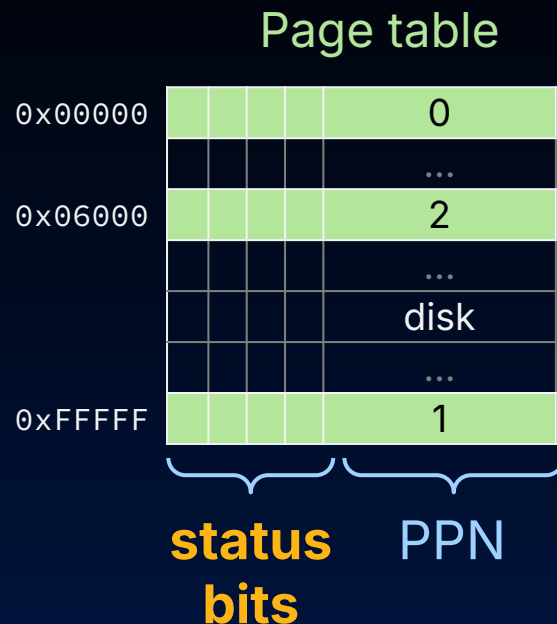
1. Is a page table a type of cache?

- One page table per process.
- One entry per virtual page number:
 - Entry has **physical page number** (if in memory)
 - Entry has **status bits**: valid, dirty, write protection (discussed later)

1. Is a page table a type of cache?

No! A page table is **NOT** a cache!!

- A page table is a lookup table!
- It does NOT have data.
- All VPNs have an entry in the page table.



What *are* Page tables?

- One page table per process.
- One entry per virtual page number:
 - Entry has **physical page number** (if in memory)
 - Entry has **status bits**: valid, dirty, write protection (discussed later)

Page table

0x00000				0
				...
0x06000				2
				...
				disk
				...
0xFFFFF				1

status bits PPN

2. Can a full page table be stored in the cache?

- Page table sizes: ~1MiB
- L1 cache: ~64KiB
- L2 cache: ~4MiB

No! Too large!

Address Translation Steps: One Disk Access

1. Program wants to access memory at a given **virtual address (VA)**.
2. OS translates **VA** to the **physical address (PA)** in memory.
 - a. Extract **virtual page number (VPN)** from VA
 - b. Look up **physical page number (PPN)** in **page table**.
 - c. Construct PA: **physical page number** + offset (offset same as in virtual address)
3. **Page Fault**: If physical page not in memory, then OS loads page from disk.
4. Read memory at the physical address and return the data to the program.

Demand paging
means possible
disk access here

Address Translation Steps: Two Mem. Accesses

1. Program wants to access memory at a given **virtual address (VA)**.
2. OS translates **VA** to the **physical address (PA)** in memory.
 - a. Extract **virtual page number (VPN)** from VA
 - b. Look up **physical page number (PPN)** in **page table**.
 - c. Construct PA: **physical page number** + offset (offset same as in virtual address)
3. **Page Fault**: If physical page not in memory, then OS loads page from disk.
4. Read memory at the physical address and return the data to the program.

Memory access 1
called

} **page table walk**

} **Demand paging**
means possible
disk access here

} Memory access 2

We have caches though.
Why can't we go faster?